

Main Report — HW06 API Testing

1. Introduction

Exercise: HW06-AI

SUT: EShop

Student ID: 23127325

Student Name: Le Gia Bao

Tools used:

- ChatGPT / Cursor
- Postman
- Newman
- GitHub Actions

2. API Selection

Three APIs were selected, with one API from each required pool:

- **API 1** — POST /api/forgot-password
- **API 2** — POST /api/cart
- **API 3** — PUT /api/orders/:id/cancel

3. API Specification and Contract Analysis

The API contracts were analyzed based on the provided api_specification.md. The analysis focused on the HTTP method, endpoint, request data, authentication requirements, expected responses, and validation rules.

3.1 API 1 — Forgot Password

- **Method:** POST
- **Endpoint:** /api/forgot-password
- **Request Body:** email
- **Success Response:** 200 OK
- **Purpose:** Generates a password-reset token for the provided email.

3.2 API 2 — Add to Cart

- **Method:** POST
- **Endpoint:** /api/cart
- **Request Body:** id, name, price, quantity

- **Purpose:** Adds a product with the specified information and quantity to the shopping cart.

3.3 API 3 — Cancel Order

- **Method:** PUT
- **Endpoint:** /api/orders/:id/cancel
- **Path Parameter:** id
- **Purpose:** Changes an order's status to canceled.
- **Business Rule:** An order can only be canceled before shipping.

4. AI Test Generation

4.1 Forgot-Password API

Endpoint: POST /api/forgot-password

AI was used to generate the initial test cases through step-by-step analysis of the API specification. The generated tests covered valid and invalid email formats, required fields, data types, boundary values, security inputs, malformed requests, and HTTP headers.

Test cases: A total of **30 test cases were generated by AI (TC-FP-001–TC-FP-030)**. After human review, **7 additional test cases (TC-FP-031–TC-FP-037)** were created to improve coverage.

AI-Generated Test Cases:

- **TC-FP-001** — Valid forgot-password request — *Positive*
- **TC-FP-002** — Valid admin email — *Positive*
- **TC-FP-003** — Email missing @ symbol — *Domain*
- **TC-FP-004** — Email missing domain — *Domain*
- **TC-FP-005** — Email missing local part — *Domain*
- **TC-FP-006** — Empty email — *Validation*
- **TC-FP-007** — Null email — *Validation*
- **TC-FP-008** — Missing email field — *Validation*
- **TC-FP-009** — Email provided as number — *Data Type*
- **TC-FP-010** — Email provided as boolean — *Data Type*
- **TC-FP-011** — Email provided as array — *Data Type*
- **TC-FP-012** — Email provided as object — *Data Type*
- **TC-FP-013** — Non-existent email — *Negative*
- **TC-FP-014** — SQL injection in email — *SEC-01*
- **TC-FP-015** — UNION SQL injection in email — *SEC-01*
- **TC-FP-016** — XSS payload in email — *SEC-02*
- **TC-FP-017** — HTML injection in email — *Security*
- **TC-FP-018** — Command injection string — *Security*

- **TC-FP-019** — Email with leading/trailing spaces — *Boundary*
- **TC-FP-020** — Email with internal space — *Domain*
- **TC-FP-021** — Uppercase email — *Domain*
- **TC-FP-022** — Email with plus alias — *Domain*
- **TC-FP-023** — Email with subdomain — *Domain*
- **TC-FP-024** — Very long email — *Boundary*
- **TC-FP-025** — Maximum-length formatted email — *Boundary*
- **TC-FP-026** — Extra unknown field — *Validation*
- **TC-FP-027** — Empty request body — *Validation*
- **TC-FP-028** — Invalid JSON format — *Malformed Input*
- **TC-FP-029** — Wrong Content-Type: text/plain — *Validation*
- **TC-FP-030** — Missing Content-Type header — *Header*

Human-Added Test Cases:

- **TC-FP-031** — Authorization header present — *Security*
- **TC-FP-032** — Invalid Authorization header — *Security*
- **TC-FP-033** — GET with wrong HTTP method — *HTTP Method*
- **TC-FP-034** — PUT with wrong HTTP method — *HTTP Method*
- **TC-FP-035** — DELETE with wrong HTTP method — *HTTP Method*
- **TC-FP-036** — Validate success response schema — *Schema*
- **TC-FP-037** — Validate error response schema — *Schema*

Human Review:

- All 30 AI-generated test cases were manually reviewed against the API specification and the SUT. The review checked the test objective, request data, expected status code, and response assertions.
- The review identified issues such as overly broad expected status codes (400 or 404), overlapping invalid-input cases, and authentication scenarios that were not clearly required by the API contract. The cases were refined where necessary before final execution.

Extension:

- TC-FP-031 and TC-FP-032 were added to verify both valid and invalid Authorization-header scenarios, ensuring that the API's security behavior was explicitly tested.
- TC-FP-033–TC-FP-035 were added to verify that unsupported HTTP methods such as GET, PUT, and DELETE are correctly rejected by the Forgot Password endpoint.
- TC-FP-036 and TC-FP-037 were added to validate the structure of both successful and error responses, rather than checking only the HTTP status code.

Test Coverage (Final): The final test suite contains **37 test cases** and covers:

- **Valid input:** TC-FP-001–TC-FP-002
- **Email format/domain partitions:** TC-FP-003–TC-FP-005, TC-FP-019–TC-FP-025
- **Required fields:** TC-FP-006–TC-FP-008, TC-FP-027
- **Data types:** TC-FP-009–TC-FP-012
- **Non-existent email:** TC-FP-013
- **SQL Injection (SEC-01):** TC-FP-014–TC-FP-015
- **XSS / injection:** TC-FP-016–TC-FP-018
- **Malformed input:** TC-FP-028–TC-FP-029
- **Header/security testing:** TC-FP-030–TC-FP-032
- **HTTP method validation:** TC-FP-033–TC-FP-035
- **Response schema validation:** TC-FP-036–TC-FP-037
- **Total:** 37 test cases, exceeding the required ≥ 35 test cases per API.

4.2 Add-to-Cart API

Endpoint: POST /api/cart

Test Cases

AI was used to generate the initial test cases through step-by-step analysis of the API specification. The generated tests covered valid cart requests, price and quantity boundaries, required fields, data types, malformed requests, product validation, duplicate items, authentication, and security-related inputs.

Test cases: A total of 37 test cases were generated by AI (TC-CART-001–TC-CART-037). After human review, 6 additional test cases (TC-CART-038–TC-CART-043) were created to improve coverage of Authorization-header formatting, special-character handling, numeric precision, type coercion, and alternative authentication schemes.

AI-Generated Test Cases:

- **TC-CART-001** — Valid cart add request — *Positive*
- **TC-CART-002** — Zero price — *Boundary / Validation*
- **TC-CART-003** — Negative price — *Boundary / Validation*
- **TC-CART-004** — Very large price exceeding maximum — *Boundary*
- **TC-CART-005** — Price with too many decimal places — *Boundary*
- **TC-CART-006** — Negative quantity — *Boundary / Validation*
- **TC-CART-007** — Zero quantity — *Boundary / Validation*
- **TC-CART-008** — Very large quantity — *Boundary*
- **TC-CART-009** — Quantity as decimal/float — *Data Type*
- **TC-CART-010** — Empty product ID — *Validation*
- **TC-CART-011** — Null product ID — *Validation*
- **TC-CART-012** — Non-numeric product ID — *Data Type*
- **TC-CART-013** — Empty product name — *Validation*
- **TC-CART-014** — Null product name — *Validation*

- **TC-CART-015** — Very long product name — *Boundary*
- **TC-CART-016** — XSS payload in product name — *SEC-02*
- **TC-CART-017** — SQL injection in product name — *SEC-01*
- **TC-CART-018** — SQL injection in ID field — *SEC-01*
- **TC-CART-019** — Missing ID field — *Required Field*
- **TC-CART-020** — Missing name field — *Required Field*
- **TC-CART-021** — Missing price field — *Required Field*
- **TC-CART-022** — Missing quantity field — *Required Field*
- **TC-CART-023** — Extra unknown field in request — *Validation*
- **TC-CART-024** — No authentication token — *Authentication*
- **TC-CART-025** — Invalid authentication token format — *Authentication*
- **TC-CART-026** — Expired authentication token — *Authentication*
- **TC-CART-027** — SQL injection via price parameter — *SEC-01*
- **TC-CART-028** — SQL injection via quantity parameter — *SEC-01*
- **TC-CART-029** — Price provided as string instead of number — *Data Type*
- **TC-CART-030** — Quantity provided as string instead of number — *Data Type*
- **TC-CART-031** — Invalid JSON format — *Malformed Input*
- **TC-CART-032** — Empty request body — *Validation*
- **TC-CART-033** — Non-existent product ID — *Negative / Resource Validation*
- **TC-CART-034** — Adding the same product twice — *Business Logic*
- **TC-CART-035** — Price mismatch with server — *Business Logic / Validation*
- **TC-CART-036** — Unicode characters in product name — *Internationalization*
- **TC-CART-037** — Duplicate cart item with same ID but different name — *Business Logic*

Human-Added Test Cases:

- **TC-CART-038** — Missing Bearer prefix in Authorization header — *Authentication*
- **TC-CART-039** — Special characters in product name — *Input Validation*
- **TC-CART-040** — Floating-point precision in price — *Numeric Precision*
- **TC-CART-041** — Very small positive price — *Boundary*
- **TC-CART-042** — Negative price via type coercion — *Data Type / Validation*
- **TC-CART-043** — No Bearer token with Basic authentication — *Authentication*

Human Review:

- All **37 AI-generated test cases** were manually reviewed against the API specification and the SUT. The review checked the test objective, request body, authentication headers, expected HTTP status code, and Postman assertions.
- The review identified several areas that required additional consideration, particularly authentication-header variations, numeric precision, type coercion, special-character input, and alternative authentication schemes. These areas were not sufficiently covered by the original AI-generated test cases.
- The AI-generated cases were retained where they represented distinct input partitions, validation conditions, security conditions, or business rules. The human-added cases

were designed to complement the existing coverage rather than duplicate the AI-generated tests.

Extension: 6 Additional Test Cases (TC-CART-038–TC-CART-043)

- **TC-CART-038:** tests a valid JWT without the required Bearer prefix. This is different from TC-CART-024: which tests the complete absence of the Authorization header.
- **TC-CART-039:** checks whether special characters such as @#\$%^&*() in the product name are handled correctly.
- **TC-CART-040:** examines floating-point precision using the value 0.30000000000000004.
- **TC-CART-041:** verifies the lower positive price boundary using 0.01.
- **TC-CART-042:** tests a negative price represented as a string ("-10.00"), covering potential type-coercion behavior.
- **TC-CART-043:** tests an unsupported Basic authentication scheme instead of the required Bearer authentication scheme.

These cases improve the robustness of the test suite by covering input and authentication variations that were not explicitly addressed in the initial AI-generated set.

Test Coverage (Final): The final test suite contains 43 test cases and covers:

- **Valid cart request:** TC-CART-001
- **Price boundary and validation:** TC-CART-002–TC-CART-005, TC-CART-040–TC-CART-042
- **Quantity boundary and numeric validation:** TC-CART-006–TC-CART-009
- **Product ID validation:** TC-CART-010–TC-CART-012, TC-CART-018–TC-CART-019
- **Product name validation:** TC-CART-013–TC-CART-017, TC-CART-036, TC-CART-039
- **Required fields:** TC-CART-019–TC-CART-022
- **Data type validation:** TC-CART-009, TC-CART-012, TC-CART-029–TC-CART-030, TC-CART-042
- **SQL Injection (SEC-01):** TC-CART-017–TC-CART-018, TC-CART-027–TC-CART-028
- **XSS / injection:** TC-CART-016
- **Malformed input:** TC-CART-031–TC-CART-032
- **Unknown fields:** TC-CART-023
- **Authentication:** TC-CART-024–TC-CART-026, TC-CART-038, TC-CART-043
- **Resource validation:** TC-CART-033
- **Cart business logic:** TC-CART-034–TC-CART-035, TC-CART-037
- **Internationalization / special characters:** TC-CART-036, TC-CART-039
- **Numeric precision:** TC-CART-040
- **Total: 43 test cases**, exceeding the required ≥35 test cases per API.

4.3 Cancel Order API

Endpoint: PUT /api/orders/:id/cancel

Test Cases: AI was used to generate the initial test cases through step-by-step analysis of the API specification. The generated tests covered valid and invalid order IDs, order-status transitions, authentication and authorization, security inputs, malformed requests, boundary values, headers, and response validation.

A total of **30 test cases were generated by AI (TC-CANCEL-001–TC-CANCEL-030)**. After human review, **5 additional test cases (TC-CANCEL-031–TC-CANCEL-035)** were created to improve coverage.

AI-Generated Test Cases:

- **TC-CANCEL-001** — Cancel pending order — *Positive*
- **TC-CANCEL-002** — Cancel confirmed order — *Business Rule*
- **TC-CANCEL-003** — Cancel shipping order — *Business Rule*
- **TC-CANCEL-004** — Cancel delivered order — *Business Rule*
- **TC-CANCEL-005** — Cancel already cancelled order — *Business Rule*
- **TC-CANCEL-006** — Cancel non-existent order — *Negative*
- **TC-CANCEL-007** — Invalid string order ID — *Data Type*
- **TC-CANCEL-008** — Negative order ID — *Boundary*
- **TC-CANCEL-009** — Zero order ID — *Boundary*
- **TC-CANCEL-010** — Float order ID — *Data Type*
- **TC-CANCEL-011** — SQL injection in order ID — *SEC-01*
- **TC-CANCEL-012** — UNION SQL injection in order ID — *SEC-01*
- **TC-CANCEL-013** — Missing authentication — *Security*
- **TC-CANCEL-014** — Invalid token format — *Security*
- **TC-CANCEL-015** — Expired token — *Security*
- **TC-CANCEL-016** — IDOR / other user's order — *SEC-03*
- **TC-CANCEL-017** — Basic authentication instead of Bearer token — *Security*
- **TC-CANCEL-018** — Missing Bearer prefix — *Security*
- **TC-CANCEL-019** — XSS payload in authorization token — *Security*
- **TC-CANCEL-020** — Very long order ID — *Boundary*
- **TC-CANCEL-021** — Null order ID — *Validation*
- **TC-CANCEL-022** — Missing order ID — *Validation*
- **TC-CANCEL-023** — Extra fields in request body — *Validation*
- **TC-CANCEL-024** — Different-role token — *Authorization*
- **TC-CANCEL-025** — Rate-limiting probe — *Security / Performance*
- **TC-CANCEL-026** — Concurrent cancellation probe — *Concurrency*
- **TC-CANCEL-027** — Invalid JSON request — *Malformed Input*
- **TC-CANCEL-028** — Token missing user context — *Security*
- **TC-CANCEL-029** — Cancellation of in-progress order — *Business Rule*
- **TC-CANCEL-030** — Special characters in order ID — *Boundary / Validation*

Human-Added Test Cases:

- **TC-CANCEL-031** — Validate successful cancellation response schema — *Schema*
- **TC-CANCEL-032** — Validate error response schema — *Schema*
- **TC-CANCEL-033** — Cancel order after refund has been processed — *Business Rule*
- **TC-CANCEL-034** — Missing Content-Type header — *Header*
- **TC-CANCEL-035** — Verify order status is changed after cancellation — *State Transition*

Human Review:

- All 30 AI-generated test cases were manually reviewed against the API specification and the SUT. The review checked the test objective, order ID, authentication headers, expected status code, business-rule assumptions, and response assertions.
- The review identified several cases where the generated expected status codes were intentionally broad, such as 400 or 404, because the exact behavior depends on how the SUT handles invalid route parameters. Authentication and authorization cases were also reviewed to distinguish authentication failures (401) from authorization or resource-access failures (403/404).
- The test cases were refined and retained where they provided meaningful coverage of input validation, security, business rules, and robustness.

Extension:

- 5 additional test cases (TC-CANCEL-031–TC-CANCEL-035) were created manually after the AI review.
- The main gaps identified were explicit response-schema validation, post-cancellation state verification, refund-related business rules, and missing Content-Type behavior. These cases were added to verify not only the HTTP status code but also the correctness of the API response and the resulting order state.
- In particular, TC-CANCEL-035 verifies the state transition after a successful cancellation, while TC-CANCEL-033 covers the business rule where a refund has already been processed.

Test Coverage (Final): the final test suite contains 35 test cases and covers:

- **Valid cancellation:** TC-CANCEL-001
- **Order-status/business-rule partitions:** TC-CANCEL-002–TC-CANCEL-005, TC-CANCEL-029, TC-CANCEL-033
- **Non-existent order:** TC-CANCEL-006
- **Order ID validation:** TC-CANCEL-007–TC-CANCEL-010, TC-CANCEL-020–TC-CANCEL-022, TC-CANCEL-030
- **SQL Injection (SEC-01):** TC-CANCEL-011–TC-CANCEL-012
- **Authentication:** TC-CANCEL-013–TC-CANCEL-015, TC-CANCEL-017–TC-CANCEL-019, TC-CANCEL-028
- **Authorization / IDOR:** TC-CANCEL-016, TC-CANCEL-024
- **Rate limiting / concurrency:** TC-CANCEL-025–TC-CANCEL-026
- **Request validation / malformed input:** TC-CANCEL-023, TC-CANCEL-027

- **Response schema validation:** TC-CANCEL-031–TC-CANCEL-032
- **Header validation:** TC-CANCEL-034
- **State transition validation:** TC-CANCEL-035
- **Total: 35 test cases**, meeting the required ≥ 35 test cases per API.

5. Test Execution And Result

5.1 Forgot Password Endpoint

The Forgot Password API was tested using Postman for test-case design and Newman for automated command-line execution. The test suite was created as a Postman collection containing positive, negative, validation, security, malformed-input, HTTP-method, header, and response-schema test cases.

Endpoint: POST /api/forgot-password

Postman Collection

- **Collection:** Forgot Password API - Newman
- **Collection Description:** 37 test cases for POST /api/forgot-password. Configure environment variables before running.
- **Total Test Cases:** 37
- **Test Execution Tool:** Newman
- **Request Method:** POST
- **Target Endpoint:** /api/forgot-password

Postman Environment

The collection was executed using the following Postman environment:

- **Base URL:** Configured through the environment variables.

Run Date and Time: Saturday, 29 August 2026, 10:01:46

The Newman Run Dashboard provided the following execution summary:

Newman Run Information	Value
Collection	Forgot Password API - Newman
Total Iterations	1

Total Assertions	37
Total Failed Tests	8
Total Skipped Tests	0
Total Run Duration	3.3s
Average Response Time	5ms
Total Data Received	3.94

The results demonstrate that the complete test suite was executed successfully by Newman. The 8 failed tests were retained as execution results for further analysis in the test summary and failure investigation sections rather than being excluded from the final report.

5.2 Add To Cart Endpoint

The Add To Cart API was tested using Postman for test-case design and Newman for automated command-line execution. The test suite was created as a Postman collection containing positive, negative, validation, security, boundary, malformed-input, authentication, and business-rule test cases.

Endpoint: POST /api/cart

Postman Collection

- **Collection:** Add To Cart API Test Collection
- **Total Test Cases:** 43
- **Test Execution Tool:** Newman
- **Request Method:** POST
- **Target Endpoint:** /api/cart

Postman Environment

The collection was executed using the following Postman environment:

- **AuthToken:** Authentication Token
- **Base URL:** Configured through the environment variables.

Run Date and Time: Saturday, 29 August 2026, 10:01:19

The Newman Run Dashboard provided the following execution summary:

Newman Run Information	Value
Collection	Add To Cart API Test Collection
Total Iterations	1
Total Assertions	43
Total Failed Tests	31
Total Skipped Tests	0
Total Run Duration	3.7s
Average Response Time	3ms
Total Data Received	2.16KB

The results demonstrate that the complete test suite was executed by Newman. All 43 requests and 43 test scripts were executed, with no skipped tests. The Newman run recorded 31 failed assertions, which are retained as execution results for further analysis in the test summary and failure investigation sections rather than being excluded from the final report.

5.3 Cancel Order Endpoint

The Cancel Order API was tested using Postman for test-case design and Newman for automated command-line execution. The test suite was created as a Postman collection containing positive, negative, validation, security, authorization, boundary, malformed-input, concurrency, and response-schema test cases.

Endpoint: PUT /api/orders/:id/cancel

Postman Collection

- **Collection:** Cancel Order API - Newman
- **Collection Description:** 35 test cases for PUT /api/orders/:id/cancel. Configure environment variables before running.
- **Total Test Cases:** 35
- **Test Execution Tool:** Newman
- **Request Method:** PUT
- **Target Endpoint:** /api/orders/:id/cancel

Postman Environment

The collection was executed using the following Postman environment:

- **Base URL:** Configured through the environment variables.
- **AuthToken:** Bearer token configured through authToken.

Run Date and Time: Saturday, 29 August 2026, 10:00:00

The Newman Run Dashboard provided the following execution summary:

Newman Run Information	Value
Collection	Cancel Order API - Newman
Total Iterations	1
Total Assertions	35
Total Failed Tests	1
Total Skipped Tests	0
Total Run Duration	3.1s
Average Response Time	7ms
Total Data Received	2.27KB

6. CI/CD Integration

6.1 CI/CD Pipeline Configuration

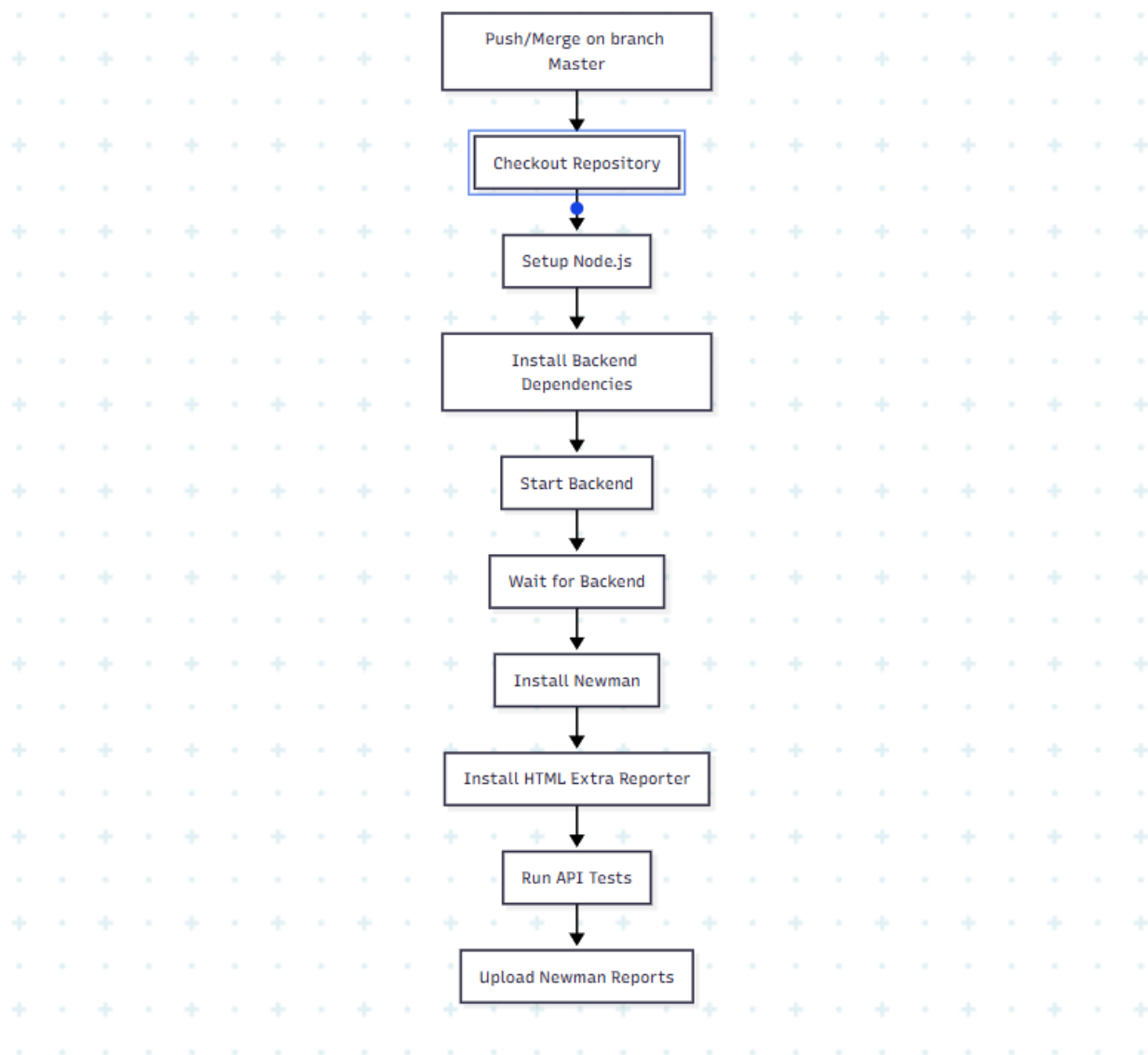
The API test suite was integrated into the project's CI/CD pipeline using **GitHub Actions**. The pipeline automatically prepares the EShop backend, starts the SUT, executes the Postman API test collection with Newman, and uploads the generated HTML test report as a workflow artifact.

The CI/CD pipeline consists of the following steps:

1. **Checkout repository** – Retrieves the latest project source code from the GitHub repository.
2. **Setup Node.js** – Configures **Node.js 24** on the GitHub Actions runner.
3. **Install backend dependencies** – Runs npm install inside eshop-sut/backend to install the backend dependencies.
4. **Start backend** – Starts the EShop backend application before executing the API tests.

5. **Wait for backend** – Checks whether the API is available at <http://localhost:3000> before continuing with the test execution.
6. **Install Newman** – Installs Newman globally to execute the Postman API collection from the command line.
7. **Install HTML Extra Reporter** – Installs newman-reporter-htmlextra to generate a detailed HTML execution report.
8. **Run API tests** – Executes the Postman collection against the running EShop API using Newman.
9. **Upload Newman reports** – Uploads the generated HTML reports as GitHub Actions artifacts so that the execution results can be reviewed after the workflow finishes.

6.2 CICD Diagram



7. AI Critique

During this assignment, AI was useful for generating a large number of API test cases and identifying common categories such as valid inputs, invalid inputs, boundary values, authentication, injection attacks, malformed requests, and response validation.

However, the generated test cases were not always accurate or sufficiently aligned with the actual System Under Test (SUT). For example, AI sometimes proposed overly broad expected status codes such as 200/400, 400/404, or 401/403, instead of determining the exact expected behavior from the API implementation and specification. Some generated cases were also redundant or tested scenarios that were not explicitly required by the API contract.

In addition, AI focused heavily on common security and input-validation patterns but could miss application-specific behavior, such as valid order-state transitions, ownership of an order, or the actual behavior of authentication middleware. This happened because AI primarily reasoned from the API description and general testing knowledge rather than observing the complete runtime behavior of the SUT. It also cannot reliably assume how a particular backend implementation handles ambiguous or unsupported inputs without executing the tests. Human review was therefore necessary to validate the generated cases, correct unrealistic expectations, and add missing scenarios.

The main principle I learned is that AI should be treated as a test-design assistant rather than an authoritative source of truth. AI can improve coverage and accelerate test-case generation, but the tester remains responsible for verifying assumptions against the specification, source code, and actual execution results. Effective collaboration with AI requires a continuous cycle of AI generation → human review → test execution → failure analysis → refinement.

8. Conclusion

This assignment demonstrated the application of systematic API testing to the EShop System Under Test (SUT), from test-case design to automated execution and CI/CD integration. Postman and Newman were used to develop and execute API test cases covering positive and negative scenarios, validation, boundary conditions, authentication, security, malformed input, HTTP methods, and response behavior.

AI significantly supported the testing process by generating initial test cases and suggesting potential edge cases. However, human review and actual execution were necessary to identify incorrect assumptions, refine expected results, remove redundant cases, and add application-specific scenarios that AI did not adequately cover. The execution results also demonstrated that having a large number of test cases does not guarantee that all tests are correct; the expected behavior must always be verified against the API specification and the actual SUT.

Finally, integrating Newman into GitHub Actions demonstrated how API tests can become part of an automated CI/CD workflow. The pipeline automatically prepares the backend, executes the API tests, generates HTML reports, and preserves the results as artifacts. The two sample pipeline runs also showed that the workflow can distinguish between a fully passing test suite and a build containing a failing API test.

Overall, the assignment provided practical experience in AI-assisted test design, API testing, automated test execution, failure analysis, and continuous integration, while reinforcing the importance of combining AI assistance with human judgment and empirical testing.

9. Assessment

	Criteria	Grade	Self-Assessed Grade
1	API 1 — full pipeline (generate + audit + extend + execute + bugs)	30	30
2	API 2 — full pipeline (same criteria)	30	30
3	API 3 — full pipeline (same criteria)	30	30
4	Agent Skills (AI-driven test generator)	10	10
	Total	100	100